

Cualidades del Software



Por
Nicolás Passerini
Carlos Lombardi
Fernando Dodino

Versión 1.0
Septiembre 2005

ÍNDICE

OBJETIVO	3
UNA ENUMERACIÓN... ..	3
ELECCIÓN DE LAS CUALIDADES	5
RELACIÓN ENTRE CUALIDADES	5
RELACIÓN CON LOS CONCEPTOS QUE CONOCEMOS.....	5
ROBUSTEZ.....	5
FLEXIBILIDAD	6
CLARIDAD	6
PARA TERMINAR	7

Objetivo

¿Por qué hacer un apunte de “Cualidades del software”? Tenemos algunos móviles:

- Definir un vocabulario común para las clases, para los exámenes, etc. eliminando ambigüedades al decir “la solución es/no es flexible”, “el diseño es/no es robusto”, etc.
- Enganchar las herramientas que vemos en la materia con las cualidades del software que generamos, y cómo llegar a conseguir esas cualidades.
- Dar un aspecto más formal a muchas reglas heurísticas que existen y que nos han acompañado a lo largo de los años en que los docentes de TADP hemos desarrollado sistemas¹.
- Entender qué puedo pedir a un sistema y qué consecuencias tiene eso que estoy pidiendo.

Tomamos como premisa que este documento no tiene preceptos morales, sino un conjunto de ideas que hacen al espíritu de esta materia y que de alguna manera intentaremos cristalizar aquí.

Una enumeración...

¿Qué cualidades aparecen en un sistema?²

Los que nos parece más importante mencionar son:

- **Robustez:** es un concepto difícil de definir, pero haremos nuestro intento: robustez = distancia al caos. Mientras menor es la distancia al caos, mayor solidez/robustez posee ese sistema. También puedo medir la robustez de un sistema en base a la tranquilidad al: 1) usar, 2) modificar una pieza de software.
- **Corrección:** un sistema es correcto si hace lo que el cliente necesita. Dicho de otra forma, un sistema es correcto si su resuelve el problema **real** que causó su desarrollo. Adaptación total a una especificación no necesariamente implica corrección ya que dicha especificación puede no ser un fiel reflejo de la realidad del problema a resolver. Dicho de otra forma: pasa todo el tiempo que el cliente te pide y/o espera algo distinto de lo que necesita (de hecho lo que dice, lo que espera y lo que necesita pueden ser los tres súper distintos entre sí).
- **Eficiencia o Performance:** tenemos dos dimensiones posibles para medir la eficiencia (tiempo/recursos) de un sistema:
 - Recursos necesarios para la construcción (tiempo de desarrollador)³
 - Recursos necesarios para la ejecución (tiempo de usuario + hardware)“Tiene mejor eficiencia el sistema que necesita menos recursos para realizar una tarea determinada”, por lo tanto deberíamos considerar ambas dimensiones a la hora de medir esta cualidad.
- **Claridad:** Se refiere a la posibilidad de entender el funcionamiento de un sistema, subsistema o una porción de código cualquiera, su objetivo y la forma de solucionar el problema; en particular por gente que no es la que lo construyó. La claridad de un módulo afecta claramente a la posibilidades de modificarlo (flexibilidad).
- **Flexibilidad:** es la capacidad que tiene un sistema para reflejar cambios percibidos en el dominio (sea por mejor comprensión del mismo o porque de verdad cambió) de una manera simple y sencilla. Conceptos relacionados son:

¹ Aclaremos que viejos son los trapos, señores.

² Todas las definiciones que daremos son materia opinable; es posible que un mismo concepto aparezca con otro nombre en alguna otra bibliografía. Lo que consideramos importante es que al lector le sirva como punto de partida de su propia opinión.

³ Hablar de recursos (siempre limitados) no implica sólo el tiempo de generar líneas de código, sino también de los recursos “ocultos” de todo desarrollo: el buen clima de trabajo, las ganas de trabajar de mis programadores, la salud de los programadores, la calidad del software que vamos construyendo, el conocimiento distribuido en el equipo de trabajo y su posibilidad de trabajar como tal, etc. Todo eso también forma parte del capital de cada proyecto, y obtener o perder cada ítem también tiene un costo/beneficio asociado (que todo manager debería evaluar).

- **Extensibilidad:** un sistema es extensible cuando pueden incorporarse nuevas características al mismo sin mayor impacto sobre las características actuales.
- **Mantenibilidad:** un sistema o desarrollo es más mantenible cuanto menor esfuerzo requiere para que el sistema siga funcionando en condiciones distintas a las originales e incluso en las originales. Entre estas tareas podríamos enumerar:
 - Pequeños cambios de funcionalidad o parametrización (aquí se relaciona con la flexibilidad)
 - Debugging
 - Posibilidad de corregir las incongruencias producidas por los propios errores del software
 - Posibilidad de hacer cualquier tipo de cambio sobre el sistema mientras este sigue funcionando.
- **Consistencia:** el sistema debe comportarse siempre de la misma manera ante un mismo evento y las tareas similares deben poder realizarse siguiendo pasos similares.
- **Simplicidad:** el sistema debe ser simple tanto en la interfaz como en la implementación. Es más importante la simplicidad en la interfaz que en la implementación.
- **Compleitud:** Un sistema es completo cuando contempla todas las posibles situaciones a darse en la práctica.
- **Encapsulamiento o Modularidad:** poder agrupar unidades funcionales me permite que el sistema sea cohesivo, reduciendo la complejidad del sistema y aumentando en cierta forma su flexibilidad.
- **Escalabilidad:** la facilidad con la que un sistema pensado originalmente para una carga determinada puede ser adaptado para soportar una carga mayor. Las aplicaciones Web nos dan una buena muestra de cuándo la escalabilidad puede ser importante para no afectar 1) la imagen del usuario que da vida a nuestro sistema, 2) la imagen corporativa del negocio que manejamos.
- **Abstracción:** un sistema debería contener buenas abstracciones de la realidad. Recordemos que el sistema no es la realidad, sino un modelo. En base a nuestras abstracciones podemos definir:
 - **Reusabilidad:** la posibilidad de utilizar un sistema construido anteriormente para resolver un problema nuevo.
 - **Genericidad:** Un sistema o subsistema es genérico cuando se puede aplicar a un conjunto de situaciones similares. En general reusabilidad y genericidad están relacionadas, un sistema muy específico difícilmente puede ser reutilizado.
- **Utilidad:** no está de más recordar que el sistema debe ser útil al usuario.
- **Seguridad:** Un sistema seguro debe impedir que agentes (personas u otros sistemas) no autorizados realicen acciones sobre el sistema.

Elección de las cualidades

Como ingenieros nuestro valor agregado pasa por tomar decisiones. En los primeros años de la carrera, nuestras decisiones pasaban por utilizar o no utilizar ifs, establecer un mecanismo de corte para un while y finalmente llevar al programa a resolver la especificación. Promediando la carrera, subimos el nivel de nuestras decisiones y tenemos que definir en un diseño si utilizaremos Decorator o Strategy, si un proceso X soportará concurrencia o no, etc. Cuando terminemos la carrera, en condiciones de ser ingenieros hechos y derechos, debemos definir cualidades subiendo el grado de abstracción.

1. Si estamos construyendo un sistema provisorio cuyo tiempo de vida estimado es de 6 meses, no necesito pensar en la mantenibilidad.
2. Si soy responsable de un sistema de control de un avión que por diseño no va a cambiar, la flexibilidad no me parecerá importante pero la performance será crítica.

Los objetivos son lo que se necesita de acuerdo a las condiciones de cada proyecto, las cualidades ayudan a lograr lo que se necesita.

Relación entre cualidades



Algunas cualidades tienen correlación positiva, es decir, se llevan bien entre sí y tener una cualidad me ayuda a tener la otra.

Ejemplos: claridad y mantenibilidad, modularidad y robustez.



Otras entran en tensión y más bien, parecen oponerse.

Ejemplos: robustez y flexibilidad⁴ (abrir vs. cerrar caminos, los deseos del usuario vs. el momento de implementar), la eficiencia de operar un sistema vs. la eficiencia de desarrollar esa eficiencia para operar el sistema (hora programador vs. hora usuario/hardware), simplicidad vs. extensibilidad.

Relación con los conceptos que conocemos

¿En qué medida los conceptos que conocemos nos ayudan a alcanzar ciertas cualidades de software (y a alejarnos de otras?)

Robustez

Tener un buen conjunto de test unitarios

Reduce el desconocimiento y el miedo (tanto para quien lo desarrolla como para quien lo usa)

Aumenta la predictibilidad de mi sistema

==> lo hace más robusto.

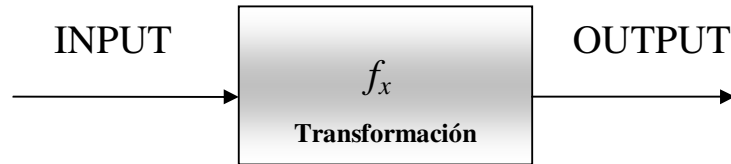
Por lo que aumenta la robustez.

El tratamiento adecuado de los errores no sólo me brinda más información cuando algo falla, sino que al prever situaciones de anomalía el sistema tolera esas fallas y se vuelve más robusto.

⁴ Didácticamente es inevitable la metáfora de la roca: la puedo ver como sólida, robusta, y como algo rígido, difícil de malear

Si creo objetos que no se modifican luego, elimino el efecto de lado y reduzco el nivel de complejidad del sistema (al transformar ciertas variables en constantes)
==> la inmutabilidad de los objetos favorece la robustez del sistema.

Más genéricamente, recordemos el concepto de transparencia referencial del paradigma funcional:



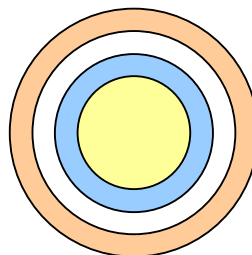
Transparencia referencial: dado 1 INPUT, siempre obtendremos la misma salida (OUTPUT), consecuencia que se obtiene por la ausencia del efecto de lado. Esto también hace que mi sistema sea más predecible (reduce la cantidad de “pruebas de escritorio” a hacer para validar mi modelo)
==> la ausencia de efecto de lado favorece la robustez del sistema

Algunas consideraciones:

- la robustez se obtiene a un costo y hay que evaluar el costo/beneficio (testing).
- finalmente uno elige cuándo ser robusto y cuándo no (OK que pocas veces no te guste).
- en la tensión robustez vs. flexibilidad está el juego de los lenguajes tipados vs. no tipados. Java valora más la robustez, Smalltalk valora más la flexibilidad.

Flexibilidad

- Pensar en conceptos como cohesión y acoplamiento, y en hacer las cosas sólo en un lugar (once and only once) impacta en varias cualidades, por ejemplo robustez y flexibilidad.
- Aplicar patrones me agrega flexibilidad, con un costo asociado que es la indirección (le resta simplicidad a mi solución⁵).
- El patrón Decorator calza con el concepto de extensibilidad⁶



Claridad

- Los lenguajes declarativos que vieron en Paradigmas de Programación nos permiten aumentar la expresividad diciendo las mismas cosas que en los lenguajes procedimentales pero con menor esfuerzo. Ser más expresivo me permite comunicar las cosas en forma más simple y clara, aunque en este aspecto también depende del lenguaje que hablen los demás interlocutores (desarrolladores).

La relación entre herramientas y cualidades no se termina aquí; simplemente preferimos dejar que el lector continúe con esta interesante tarea.

⁵ Otro debate que surge a partir de esta idea es si los patrones de diseño me agregan o me quitan claridad en la solución

⁶ Si bien la metáfora “oficial” del Decorator es la mamushka, una alternativa válida bien podría ser la cebolla

Para terminar

Todo esto que escribimos sólo definen cosas que nos gustaría que nuestros sistemas tengan, pero al final de la historia tenemos una *moraleja*: no hay un proceso mágico que haga que mi proyecto sea exitoso, sino que eso depende de que todos tomemos buenas decisiones. El proceso me tiene que ayudar llevándome por un camino que me obligue a hacerme las preguntas correctas en los momentos correctos. Esas preguntas deberían abrirme la cabeza para tener más factores a mano a la hora de decidir.

Lo que me puede dar la facultad, y que está bueno llevarse, son criterios / categorías o formas de pensar o abordar los problemas, dicho de otra forma, tener en nuestra valija un set de herramientas que nos permita encontrar las soluciones que los usuarios nos piden diariamente.



Le dejamos al lector para el fin del cuatrimestre la posibilidad de que complete la lista de herramientas que TADP le va a proveer